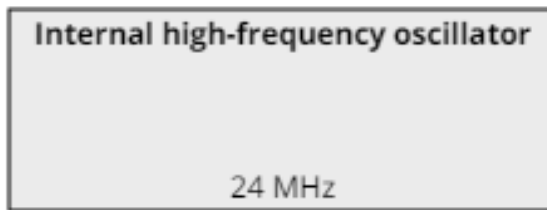


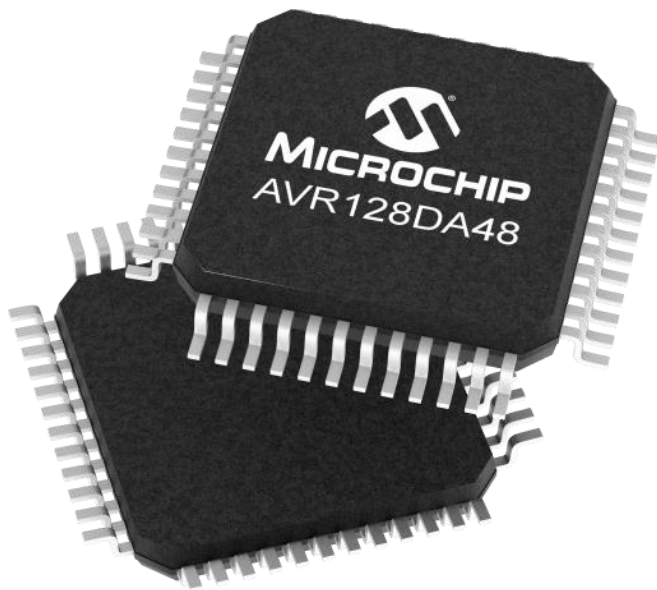
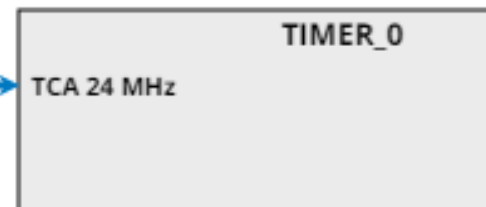
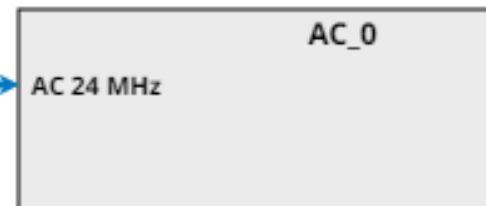
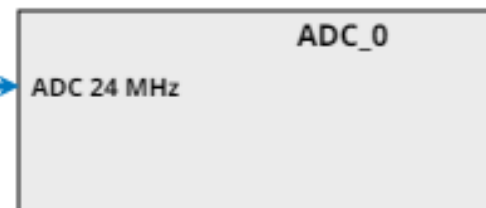
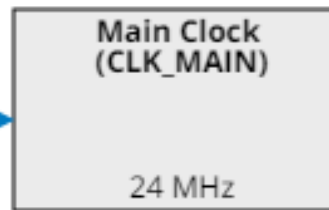
“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 30 - Final Project

OSCILLATORS



SOURCES



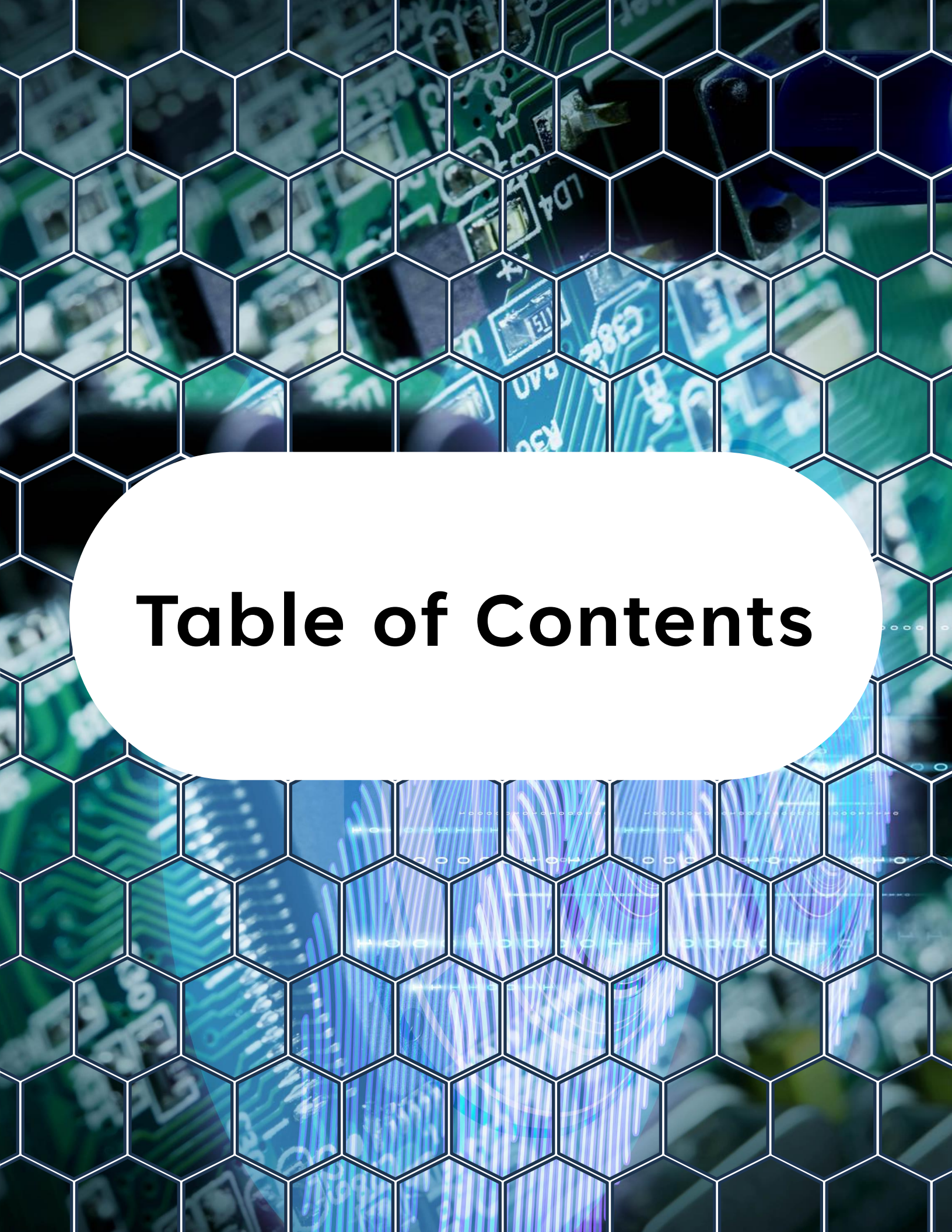
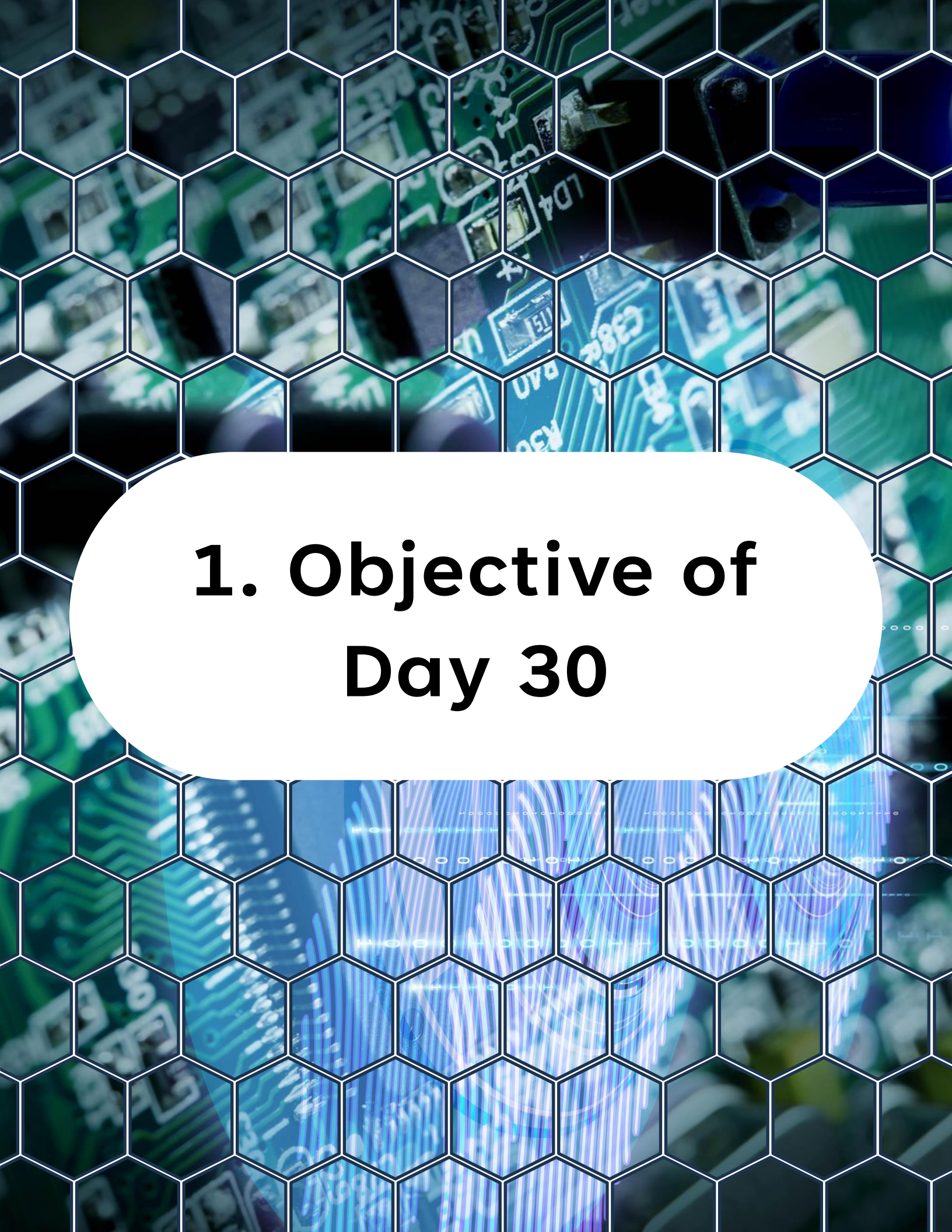


Table of Contents

Table of Contents

1. Objective of Day 30
2. Capstone Project Options
3. Required Architecture
4. Core Design Requirements
5. Example Implementation
6. Documentation Requirements
7. What a Good Final Project Looks Like
8. Common Mistakes to Avoid
9. Final Reflection



1. Objective of Day 30

1. Objective of Day 30

This is the final day of the course.

Until now, you have learned:

- Individual peripherals (ADC, TCA, USART, RTC, AC, EVSYS)
- System-level concepts (interrupts, scheduling, buffers, drivers)
- Architecture patterns (state machines, cooperative scheduling)

Today, you will bring everything together into a **complete embedded system**.

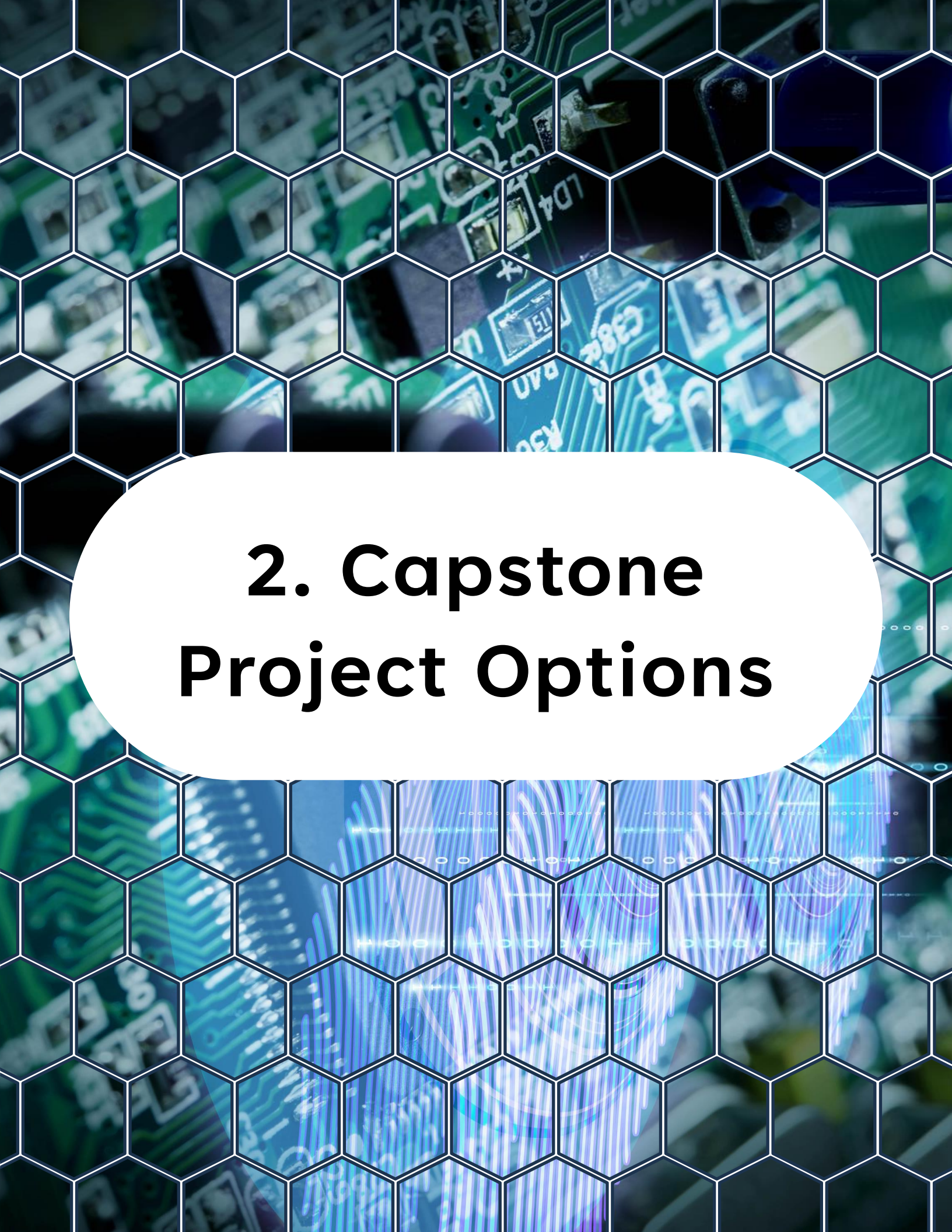
This is your **capstone project**.

1. Objective of Day 30

By the end of today, you will:

- Design a full embedded application
- Implement clean architecture
- Use register-level drivers
- Apply event-driven and cooperative design
- Produce production-quality firmware

This is where learning becomes engineering.



2. Capstone Project Options

2. Capstone Project Options

You can choose one of the following projects. Each one is designed to exercise everything you learned.

Option 1 - Smart Analog Data Logger

Description

A system that:

- Periodically samples an analog signal
- Stores or streams data
- Uses time-based acquisition
- Applies filtering or averaging

Features

- ADC sampling (periodic or event-driven)
- RTC for timing

2. Capstone Project Options

- USART for data output
- Optional EEPROM for configuration

Option 2 - PWM-Controlled Power Module

Description

A system that:

- Reads an analog input
- Controls a PWM output
- Implements a basic control loop

Features

- ADC input (sensor or potentiometer)
- TCA PWM output
- Threshold detection using AC (optional)
- Fast reaction using EVSYS (advanced)

2. Capstone Project Options

Option 3 - Serial-Controlled Measurement Device

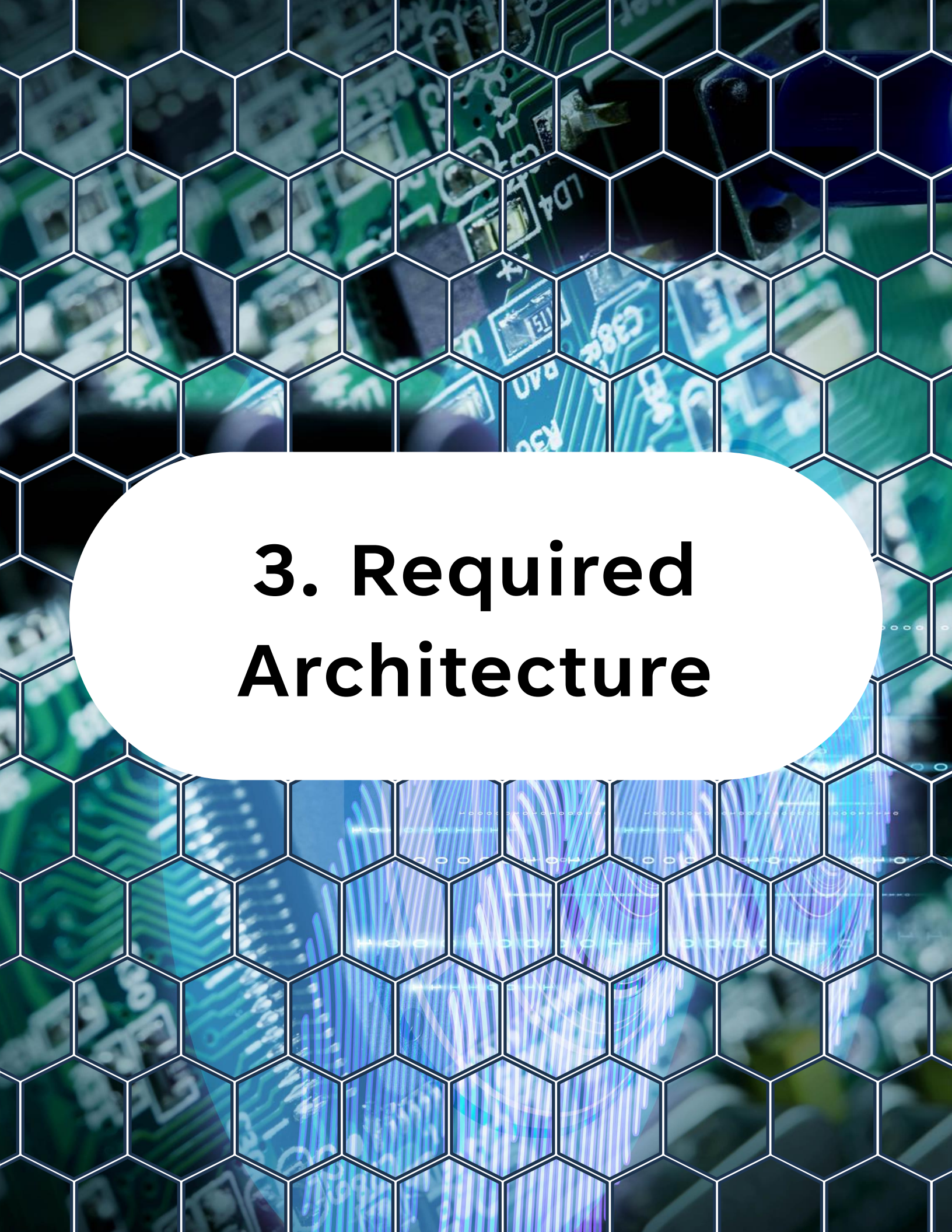
Description

A system that:

- Receives commands over USART
- Measures analog values
- Responds with formatted data

Features

- USART RX/TX with FIFO buffer
- Command parser (state machine)
- ADC measurements
- Configurable behavior



3. Required Architecture

3. Required Architecture

Your project must follow a clean structure.

3.1 Project Layout

/project

main.c

adc.c / adc.h

usart.c / usart.h

timer.c / timer.h

scheduler.c / scheduler.h

fifo.c / fifo.h

3. Required Architecture

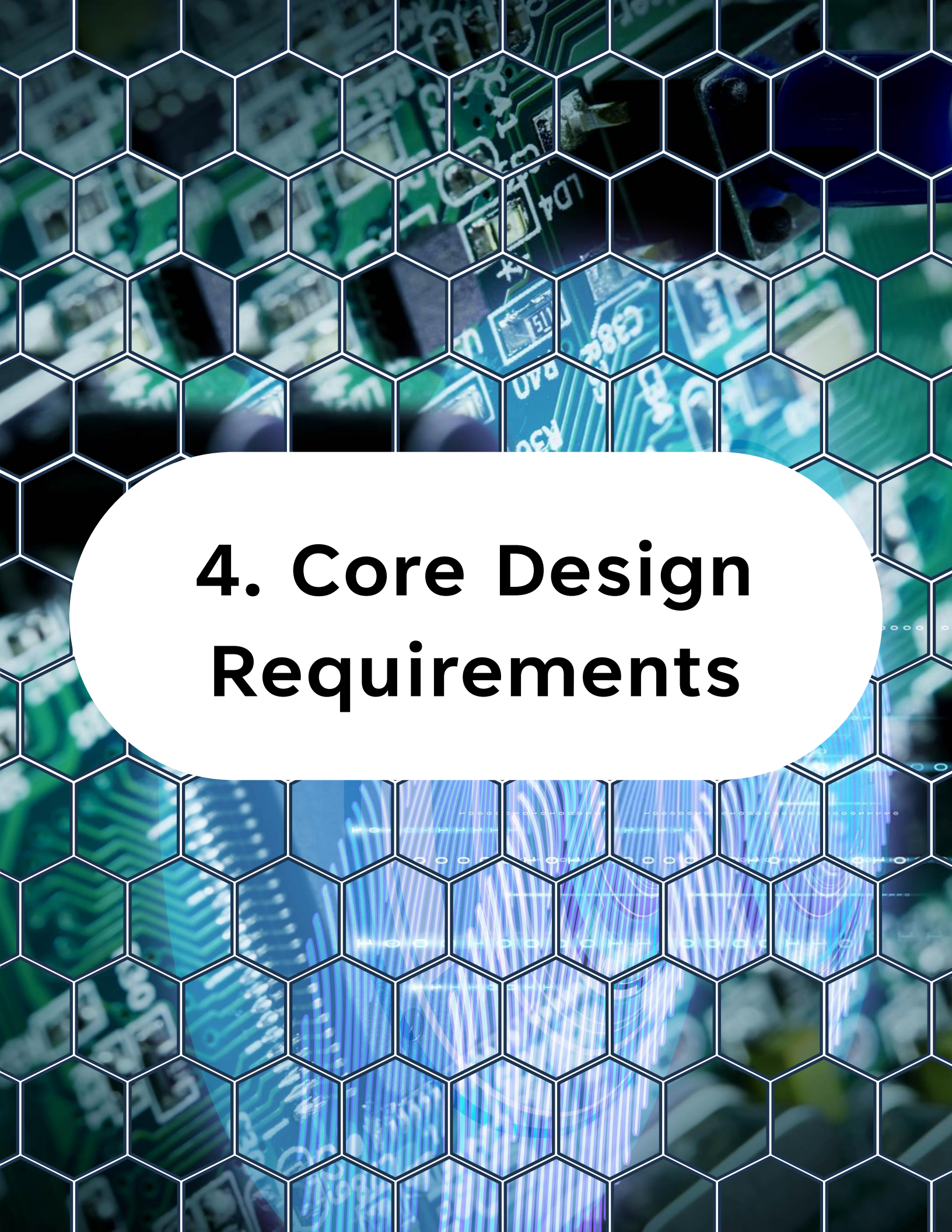
3.2 Layer Separation

Application Layer

Drivers (ADC, TCA, USART, RTC)

Hardware (Registers)

Application must not directly access registers.



4. Core Design Requirements

4. Core Design Requirements

4.1 Cooperative Scheduling

Use:

- Super-loop architecture
- Timer-based scheduling
- Non-blocking tasks

No `delay_ms()` allowed.

4.2 State Machines

At least one module must use a state machine.

Examples:

- Command parser
- Control loop
- Measurement sequence

4. Core Design Requirements

4.3 Event-Driven Behavior

Use:

- Interrupts
- Flags
- Optional EVSYS

Avoid constant polling.

4.4 Buffering

Use FIFO or buffers for:

- USART RX
- Data handling

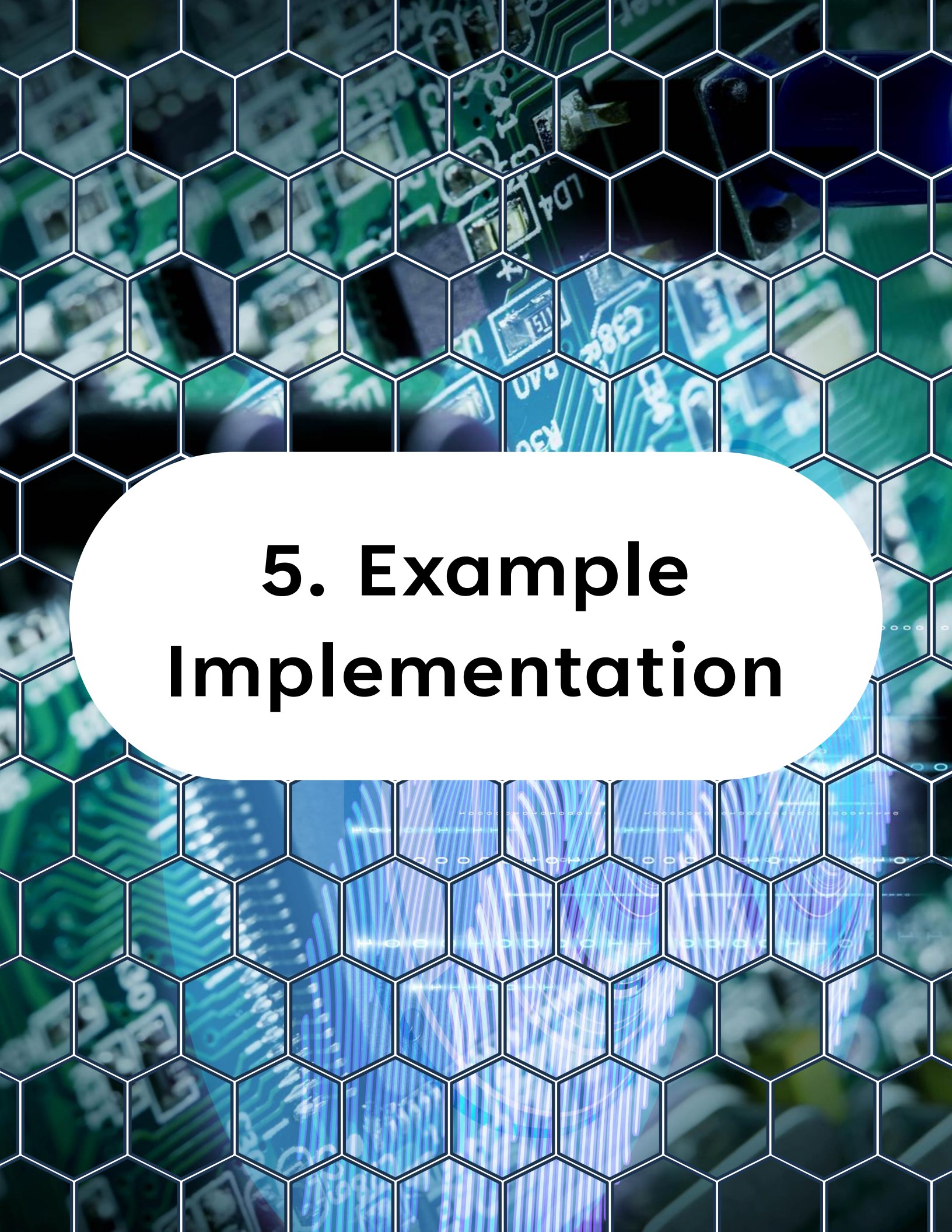
This ensures decoupling.

4. Core Design Requirements

4.5 Safe Data Sharing

Use:

- volatile variables
- Atomic access where needed



5. Example Implementation

5. Example Implementation

Serial-Controlled Measurement Device

This example demonstrates a full integration.

5.1 System Behavior

- Receive command over UART: "R"
- Perform ADC measurement
- Send value back over UART
- Use FIFO for RX
- Use cooperative scheduler

5.2 FIFO (Simplified)



```
1  #define FIFO_SIZE 32
2
3  typedef struct
4  {
5      uint8_t buffer[FIFO_SIZE];
6      volatile uint8_t head;
7      volatile uint8_t tail;
```

5. Example Implementation

5.2 FIFO (Simplified)

```
1  #define FIFO_SIZE 32
2
3  typedef struct
4  {
5      uint8_t buffer[FIFO_SIZE];
6      volatile uint8_t head;
7      volatile uint8_t tail;
8  } fifo_t;
9
10 static fifo_t rx_fifo;
```

5.3 USART RX ISR

```
1  ISR(USART0_RXC_vect)
2  {
3      // Read the received data from the USART data register
4      uint8_t data = USART0.RXDATA1;
5
6      // Calculate the next head index
7      uint8_t next = (rx_fifo.head + 1) % FIFO_SIZE;
8
9      // Check if the FIFO is not full before adding the new data
10     if (next != rx_fifo.tail)
11     {
12         rx_fifo.buffer[rx_fifo.head] = data;
```

5. Example Implementation

5.3 USART RX ISR



```
1  ISR(USART0_RXC_vect)
2  {
3      // Read the received data from the USART data register
4      uint8_t data = USART0.RXDATA1;
5
6      // Calculate the next head index
7      uint8_t next = (rx_fifo.head + 1) % FIFO_SIZE;
8
9      // Check if the FIFO is not full before adding the new data
10     if (next != rx_fifo.tail)
11     {
12         rx_fifo.buffer[rx_fifo.head] = data;
13         rx_fifo.head = next;
14     }
15 }
```

5.4 Command State Machine



```
1  typedef enum
2  {
3      CMD_IDLE,
4      CMD_RECEIVED
5  } cmd_state_t;
6
7  static cmd_state_t cmd_state = CMD_IDLE;
```

5. Example Implementation

5.5 Command Task



```
1 static void task_command(void)
2 {
3     if (rx_fifo.head != rx_fifo.tail)
4     {
5         uint8_t data = rx_fifo.buffer[rx_fifo.tail];
6         rx_fifo.tail = (rx_fifo.tail + 1) % FIFO_SIZE;
7
8         if (data == 'R')
9         {
10             cmd_state = CMD_RECEIVED;
11         }
12     }
13 }
```

5.6 ADC Task



```
1 static uint16_t adc_value = 0;
2
3 static void task_adc(void)
4 {
5     if (cmd_state == CMD_RECEIVED)
6     {
7         ADC0.COMMAND = ADC_STCONV_bm;
8
9         while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
10        {
```

5. Example Implementation

5.6 ADC Task



```
1 static uint16_t adc_value = 0;
2
3 static void task_adc(void)
4 {
5     if (cmd_state == CMD_RECEIVED)
6     {
7         ADC0.COMMAND = ADC_STCONV_bm;
8
9         while (!(ADC0.INTFLAGS & ADC_RESRDY_bm))
10        {
11        }
12
13        ADC0.INTFLAGS = ADC_RESRDY_bm;
14
15        adc_value = ADC0.RES;
16
17        cmd_state = CMD_IDLE;
18    }
19 }
```

5.7 USART TX Function



```
1 static void usart_send_uint(uint16_t value)
2 {
3     char buf[6];
4     uint8_t i = 0;
```

5. Example

Implementation

5.7 USART TX Function

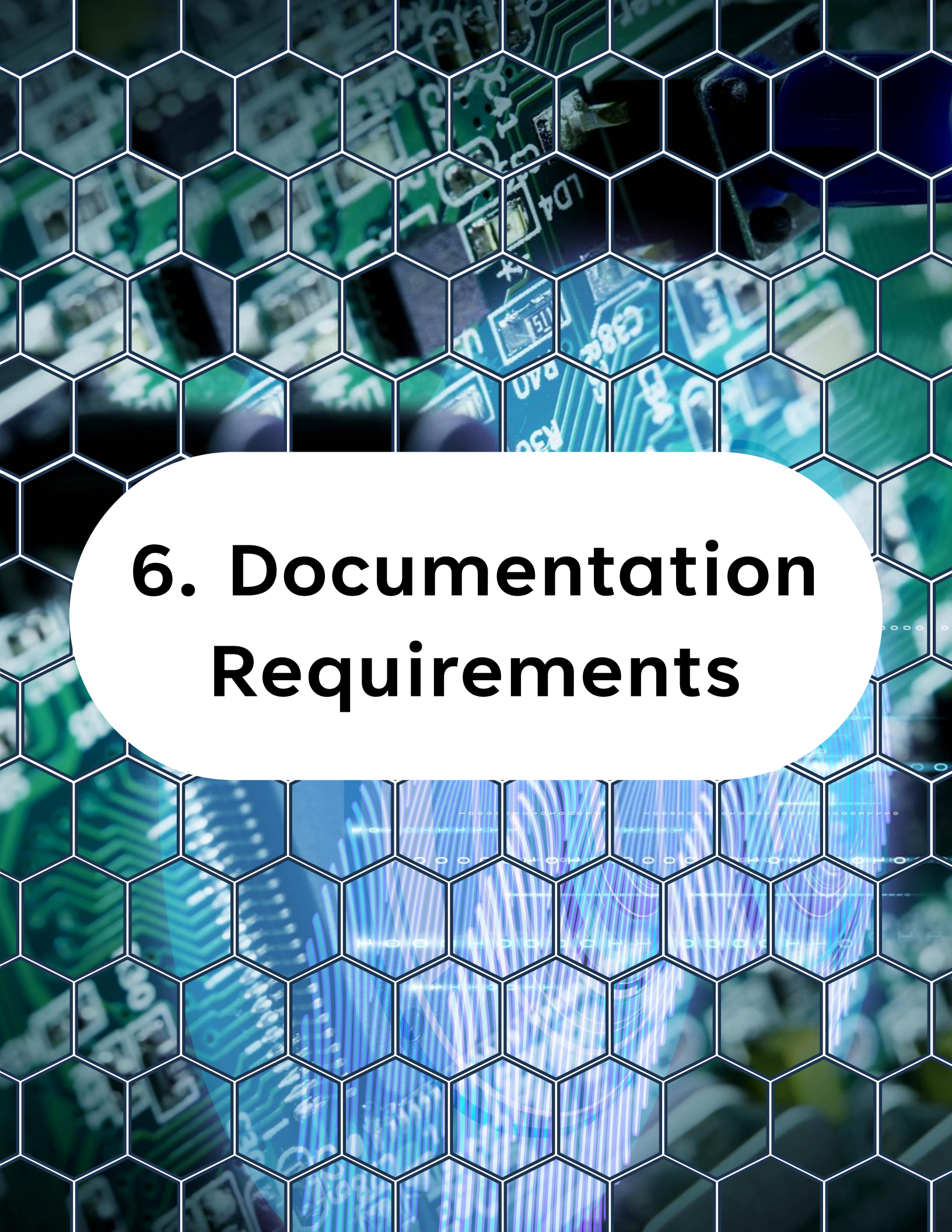
```
1 static void usart_send_uint(uint16_t value)
2 {
3     char buf[6];
4     uint8_t i = 0;
5
6     if (value == 0)
7     {
8         USART0.TXDATA1 = '0';
9         return;
10    }
11
12    while (value > 0)
13    {
14        buf[i++] = '0' + (value % 10);
15        value /= 10;
16    }
17
18    while (i > 0)
19    {
20        while (!(USART0.STATUS & USART_DREIF_bm));
21        USART0.TXDATA1 = buf[--i];
22    }
23
24    while (!(USART0.STATUS & USART_DREIF_bm));
25    USART0.TXDATA1 = '\n';
26 }
```

5. Example Implementation

5.8 Main Loop (Scheduler)



```
1  int main(void)
2  {
3      ADC0_init();
4      USART0_init();
5
6      sei();
7
8      while (1)
9      {
10         task_command();
11         task_adc();
12
13         if (adc_value)
14         {
15             usart_send_uint(adc_value);
16             adc_value = 0;
17         }
18     }
19 }
```



6. Documentation Requirements

6. Documentation Requirements

Your project must include:

- Clear function descriptions
- Module separation
- Consistent naming
- Comments explaining logic (not obvious code)

Example:



```
1  /**
2   * @brief Initialize ADC for single-ended conversion.
3   */
```



7. What a Good Final Project Looks Like

7. What a Good Final Project Looks Like

A strong submission will:

- Compile cleanly
- Have clear module separation
- Avoid blocking delays
- Use proper scheduling
- Handle data safely
- Be easy to extend



8. Common Mistakes to Avoid

8. Common Mistakes to Avoid

- Mixing register code in main
- Using blocking delays
- Not using buffers for communication
- Overcomplicating logic
- Ignoring architecture



9. Final Reflection

9. Final Reflection

If you reached this point, you have learned:

- Register-level programming
- Peripheral control
- Event-driven design
- Scheduling strategies
- System integration
- Firmware architecture

You are no longer just writing code.

You are designing embedded systems.

This course was not about memorizing registers.

It was about understanding how hardware and software interact. From now on, when you look at a datasheet, you will not see complexity. You will see possibilities.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>